

UCS645
PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 1:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

LAB 1

1. DAXPY LOOP:

```
1. #include <iostream>
2. #include <omp.h>
3. #include <vector>
4.
5. using namespace std;
6.
7. #define N 65536 //fixed the size of the vectors
8.
9. int main() {
10.     omp_set_num_threads(16);
11.     vector<double> X(N), Y(N); //double precision vectors
12.     double a = 2.5;
13.
14.     for(int i = 0; i < N; i++) {
15.         X[i] = i * 1.0;
16.         Y[i] = i * 2.0;
17.     }
18.     double start = omp_get_wtime();
19.     #pragma omp parallel for
20.     for(int i = 0; i < N; i++) {
21.         X[i] = a * X[i] + Y[i];
22.     }
23.
24.     double end = omp_get_wtime();
25.
26.     cout << "Time taken: "
27.         << end - start << " seconds" << endl;
28.
29.     return 0;
30. }
```

```

Performance counter stats for './daxpy_loop':

    108.61 msec task-clock:u          #    8.751 CPUs utilized
         0      context-switches:u    #    0.000 /sec
         0      cpu-migrations:u      #    0.000 /sec
        429     page-faults:u         #    3.950 K/sec
   383327888    cycles:u              #    3.529 GHz
   50256646    stalled-cycles-frontend:u #   13.11% frontend cycles idle
   32248963    instructions:u        #    0.08 insns per cycle
                                     #    1.56 stalled cycles per insn
   8921289     branches:u            #   82.142 M/sec
    14896      branch-misses:u       #    0.17% of all branches

    0.012410864 seconds time elapsed

    0.078621000 seconds user
    0.002620000 seconds sys

```

INFERENCE:

The OpenMP DAXPY program executes correctly in parallel and utilizes multiple CPU cores, but its performance is limited by memory access rather than computation. perf stat shows very low instructions per cycle and high stall cycles, indicating a memory-bound workload. Increasing threads beyond the number of physical cores leads to performance degradation due to thread management and memory contention. gprof fails to report meaningful function timings because the program executes too quickly and is heavily optimized, making it unsuitable for profiling short OpenMP kernels. I have 8 max physical threads and 16 logical threads it was working well till 8 threads but when I did beyond it the performance degraded.

2. Matrix Multiply:

```

1. #include <iostream>
2.
3. #include <omp.h>
4.
5. using namespace std;
6.
7. #define N 1000
8.
9. int main() {
10.     static int A[N][N], B[N][N], C[N][N];
11.     double start, end;
12.
13.     // Initialize matrices
14.     for(int i = 0; i < N; i++) {
15.         for(int j = 0; j < N; j++) {
16.             A[i][j] = 1;
17.             B[i][j] = 1;
18.         }

```

```

19.     }
20.
21.     start = omp_get_wtime();
22.
23.     // 2D threading: parallelize i and j loops
24.     #pragma omp parallel for collapse(2)
25.     for(int i = 0; i < N; i++) {
26.         for(int j = 0; j < N; j++) {
27.             C[i][j] = 0;
28.             for(int k = 0; k < N; k++) {
29.                 C[i][j] += A[i][k] * B[k][j];
30.             }
31.         }
32.     }
33.
34.     end = omp_get_wtime();
35.
36.     cout << "2D Threading Time: "
37.          << end - start << " seconds" << endl;
38.
39.     return 0;
40. }
41.

```

```

● sujal@kernel69:~/workspace/UCS645/LAB 1$ g++ -O3 -fopenmp -pg matrix_multiply.cpp -o matrix_multiply
● sujal@kernel69:~/workspace/UCS645/LAB 1$ ./matrix_multiply
2D Threading Time: 0.194808 seconds

```

Performance counter stats for './matrix_multiply':

4763.12 msec	task-clock:u	#	14.109 CPUs utilized
0	context-switches:u	#	0.000 /sec
0	cpu-migrations:u	#	0.000 /sec
3097	page-faults:u	#	650.204 /sec
18826766650	cycles:u	#	3.953 GHz
140775956	stalled-cycles-frontend:u	#	0.75% frontend cycles idle
7836192067	instructions:u	#	0.42 insn per cycle
		#	0.02 stalled cycles per insn
270754851	branches:u	#	56.844 M/sec
1033721	branch-misses:u	#	0.38% of all branches

0.337594739 seconds time elapsed

4.693619000 seconds user

0.038758000 seconds sys

INFERENCE:

The parallel matrix multiplication using 2D threading shows effective utilization of CPU cores, with perf stat reporting ~14 CPUs utilized, confirming good parallel workload distribution across both matrix dimensions. The higher instructions per cycle (0.42 IPC) and very low frontend stalls indicate better computational intensity compared to simple vector operations. The increased execution time compared to DAXPY is expected due to higher arithmetic complexity. gprof again fails to provide meaningful function-level profiling because the computation is dominated by optimized parallel loops and executes too quickly for sampling-based profiling, demonstrating that hardware counter-based profiling (perf) is more suitable for analyzing parallel matrix computations.

3. Calculation of π :

```
1  #include <iostream>
2  #include <omp.h>
3
4  using namespace std;
5
6  static long num_steps = 100000000;
7
8  int main() {
9      double step = 1.0 / num_steps;
10     double sum = 0.0;
11     double pi;
12
13     // Parallel loop with reduction
14     #pragma omp parallel for reduction(+:sum)
15     for (long i = 0; i < num_steps; i++) {
16         double x = (i + 0.5) * step;
17         sum += 4.0 / (1.0 + x * x);
18     }
19
20     pi = step * sum;
21
22     cout << "Approximate value of Pi = " << pi << endl;
23     return 0;
24 }
25
```

```

● sujal@kernel69:~/workspace/UCS645/LAB 1$ /usr/lib/linux-tools/6.8.0-94-generic/perf stat ./pi
Approximate value of Pi = 3.14159

Performance counter stats for './pi':

      356.62 msec task-clock:u          #    11.931 CPUs utilized
           0      context-switches:u    #     0.000 /sec
           0      cpu-migrations:u      #     0.000 /sec
          170      page-faults:u         #    476.701 /sec
    1299196749      cycles:u             #     3.643 GHz
       72918696      stalled-cycles-frontend:u    #     5.61% frontend cycles idle
    1243579648      instructions:u       #     0.96  insn per cycle
                                           #    0.06  stalled cycles per insn
       112252111      branches:u        #    314.769 M/sec
          15695      branch-misses:u     #     0.01% of all branches

    0.029889871 seconds time elapsed

    0.306953000 seconds user
    0.042830000 seconds sys

```

INFERENCE:

The parallel π computation shows high computational efficiency with effective utilization of CPU cores (~12 CPUs). perf stat reports a high instructions-per-cycle value (0.96 IPC) and very low stall and branch-miss rates, indicating a compute-bound workload with minimal memory and control-flow overhead. This contrasts with memory-bound kernels like DAXPY, demonstrating that OpenMP achieves better scalability and efficiency for computation-intensive tasks. gprof again provides no useful function-level breakdown because the optimized parallel loop executes too quickly for sampling-based profiling.

Summary WITHOUT using Linux tools (based on timing & behavior):

- **DAXPY (Vector operation):**
Execution time is very small, and increasing the number of threads beyond physical cores does not improve performance. The workload is too lightweight, so parallel overhead dominates.
- **Matrix Multiplication (1D & 2D threading):**
Execution time is higher due to increased computation. 2D threading performs better than 1D threading because work is evenly distributed across rows and columns, improving parallel efficiency.
- **π Computation:**
Shows good speedup with multiple threads since the computation involves heavy arithmetic and minimal memory access, making it well-suited for parallel execution.

Summary WITH Linux tools (perf, gprof):

- **DAXPY:**
perf shows very low IPC and high stall cycles, indicating a memory-bound workload (The CPU is **ready to work**, but it keeps **waiting for data** to arrive from cache or RAM). gprof fails to capture meaningful data due to very short execution time.
- **Matrix Multiplication:**
perf reports high CPU utilization and improved IPC compared to DAXPY, confirming better computational intensity. 2D threading achieves more balanced core usage. gprof again provides limited insight due to optimized parallel execution.
- **π Computation:**
perf shows high IPC, low stalls, and low branch mispredictions, confirming a compute-bound workload with excellent parallel efficiency. gprof is ineffective because the loop is fast and highly optimized.